

Code of the Day

- What's does the following kernel do?
 - Assume we launch `kernel<<<1, 256>>>` on `input = {-1, -2, -3, -4, -5, ..., -256}`, `*res = 0`

```
__global__ void kernel(int *input, int *res) {  
    int tid = threadIdx.x;  
    atomicAdd(res, input[tid]);  
}
```

- What is the problem of this implementation?
 - While `atomicAdd` prevents race conditions for updating `*res`, this causes severe serialization because only one thread can perform the atomic operation at a time
 - This bottleneck results in low performance, as most threads will be stalled waiting for the atomic operation to complete

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

Reduction

04/14/2025

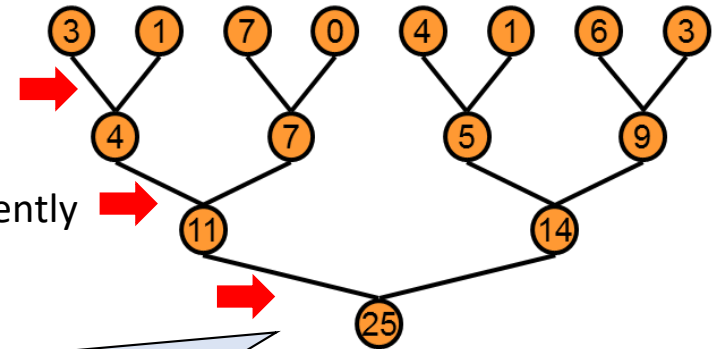
Before we get started...

- Quick overview, things discussed last time
 - Atomic operations
 - GPU occupancy – a good proxy to measure GPU performance
- Purpose of today's lecture:
 - GPU-parallel reduction kernel
- Miscellaneous
 - Programming assignment #7 is released – due **Friday 5/2 at 23:50 PM**
 - Take-home final exams will take place during the week of 4/25 – 5/4
 - In-class final project presentation take place on 4/25, 4/28, 5/2 – Please sign up [here](#)
 - You will need to upload your slides by noon on your scheduled presentation day
 - Link will be available on Canvas

Parallel Reduction in CUDA

- Parallel Reduction: Common and important data-parallel primitive
 - Ex: Used to compute the norm of a large vector
 - Ex: Used to sum up all the entries in an array

- Assume you a large array – 100,000 entries and beyond
 - Basic Idea: binary tree where each thread does two values concurrently
 - At each level, we reduce the workload by a factor of two



Only 3 parallel iterations (levels or stages) needed to reduce 8 numbers – otherwise 7 iterations needed for sequential reduction

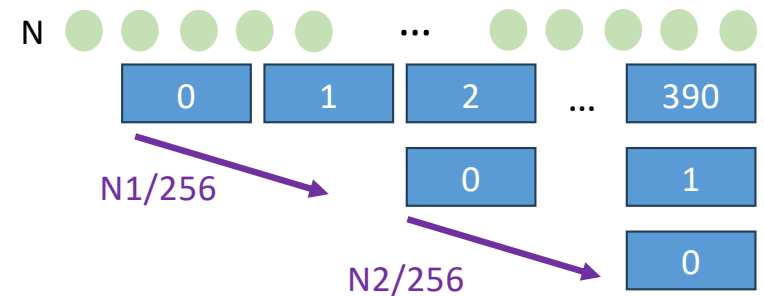
- Q: How do we communicate partial results between blocks of threads?

Problem: Global (Inter-block) Synchronization is Missing in CUDA

- If we could synchronize all thread blocks, we could easily reduce large arrays on the GPU
 - Do parallel iterations within blocks
 - Each block iteratively runs one-level (or one-stage) of the reduction tree in parallel and then sync
 - Once all blocks reach sync, continue to the next stage of the reduction tree and so on
- But CUDA has no global synchronization
 - Recall there are two scheduling levels: device and block levels
 - Each block is assigned to one SM, and different SMs are highly parallel to each other
 - There is no way to synchronize the execution across different SMs
 - We can only synchronize threads that belong to the same block using `__syncthreads()`
- Solution: **decompose the workload into multiple kernels**
 - Kernel launch serves as a global synchronization point across different blocks (inter-block sync)

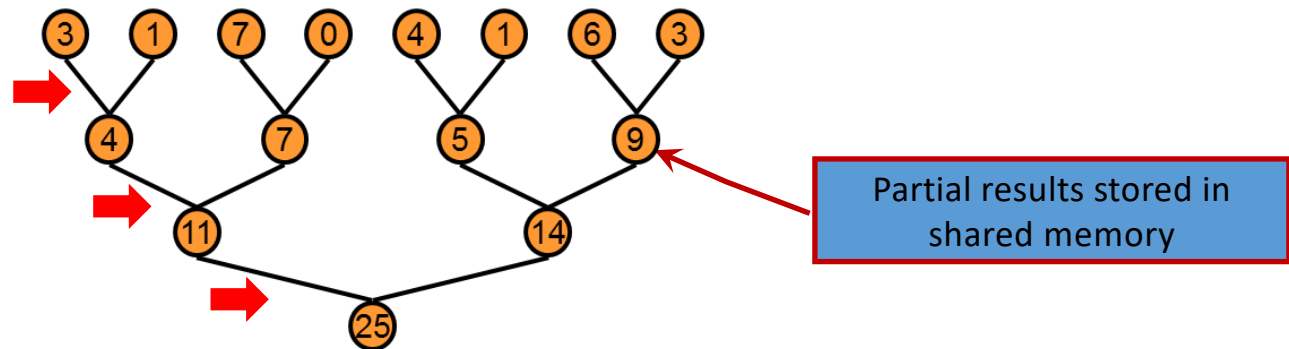
Multiple Kernel Calls to Perform Parallel Reduction

- Assume you launch a 1D grid in which each 1D block has 256 threads, and the number of elements in the array is $N=100,000$, then:
 - $\text{std::ceil}(N/256) = 391$ or $\lceil (N+255)/256 \rceil = 391$ blocks needed
- For the first stage, you launch 391 blocks of 256 threads
 - Each block reduce its assigned partition to one partial result
 - At the end of this stage you still have to operate on 391 elements
- For the second stage, you launch two blocks of 256 threads
 - At the end of this stage you only have to operate on two elements
- For the third and last stage, you launch one block of 32 threads (this is the minimum warp size)
 - Only 2 of the 32 threads are doing useful thing
- Bottom line:** after the first stage, each subsequent stage operates on a number of entries equal to the number of blocks in the previous stage

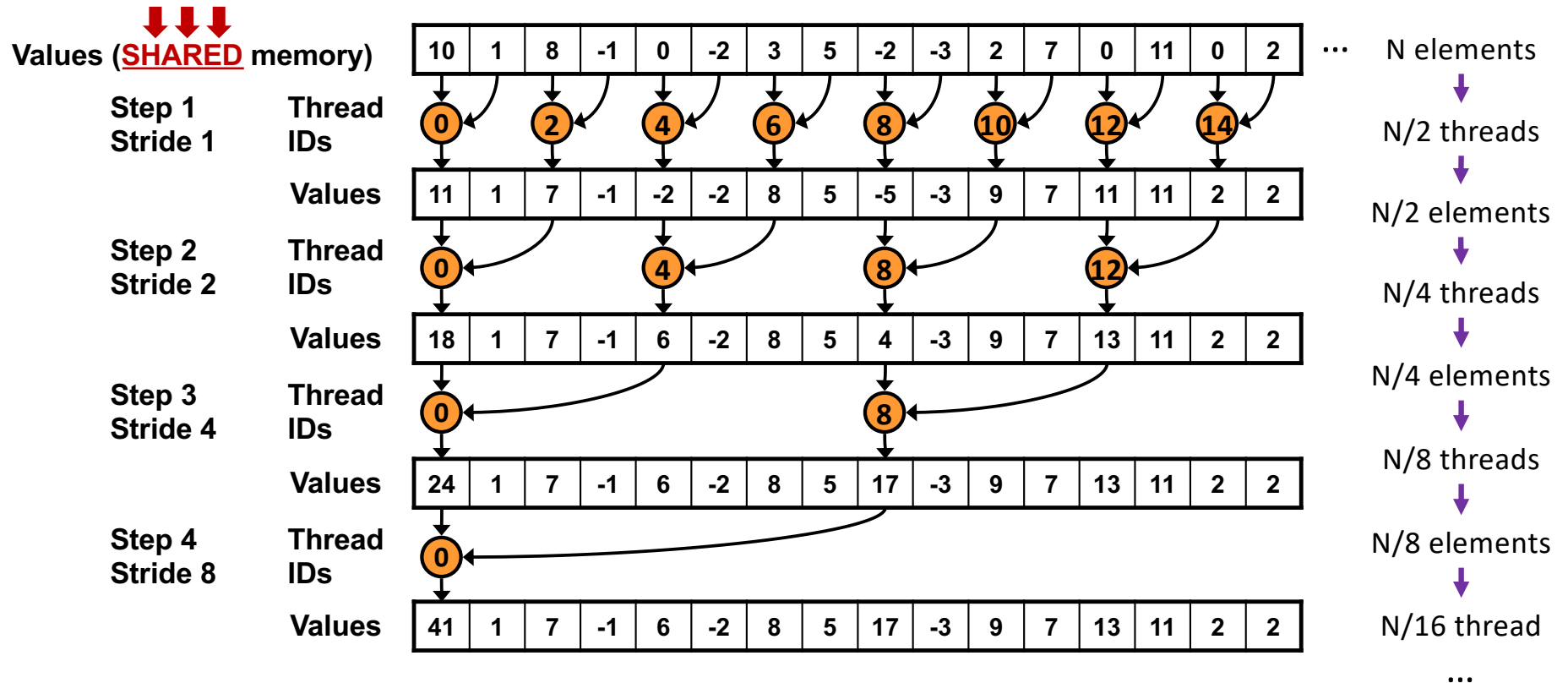


Vector Reduction: 30,000 Feet Perspective

- At the block level: Bring data in **shared memory**, then start adding in parallel
 - Why can we leverage shared memory? – each reduction iteration produce temporary/partial values that will be used in the subsequent reduction iterations
- As we progress, fewer and fewer threads within a block will participate



Reduction #1: Interleaved Addressing



Reduction #1: Interleaved Addressing

```
__global__ void reduce1(int *g_idata, int *g_odata) {  
    extern __shared__ int sdata[];  
  
    // each thread loads one element from global to shared mem  
    unsigned int tid = threadIdx.x;  
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;  
    sdata[tid] = g_idata[i];  
    __syncthreads();
```

```
    // do reduction in shared mem  
    for(unsigned int s=1; s < blockDim.x; s *= 2) {  
        if (tid % (2*s) == 0) {  
            sdata[tid] += sdata[tid + s];  
        }  
        __syncthreads();  
    }
```

Problem: highly divergent warps are inefficient, and % operator is very slow

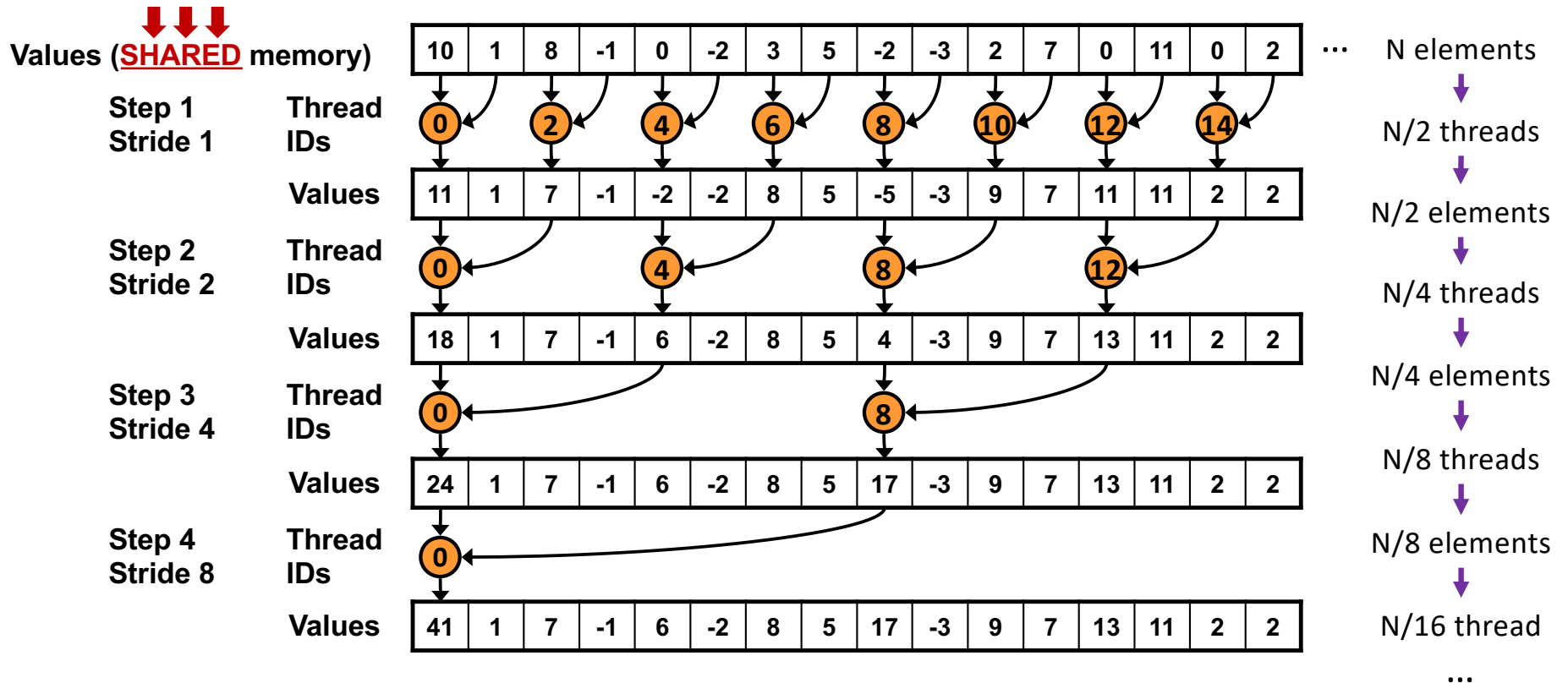
```
    // write result for this block to global memory  
    if (tid == 0) g_odata[blockIdx.x] = sdata[0];  
}
```



Performance for 4 Million element reduction

	Time (2^{22} ints)	Bandwidth
Kernel 1: interleaved addressing with divergent branching	8.054 ms	2.083 GB/s

Reduction #1: Interleaved Addressing → Thread Divergence ...



Problem: lots of thread divergence - in stage s, only threads divisible to 2^s get to do work.

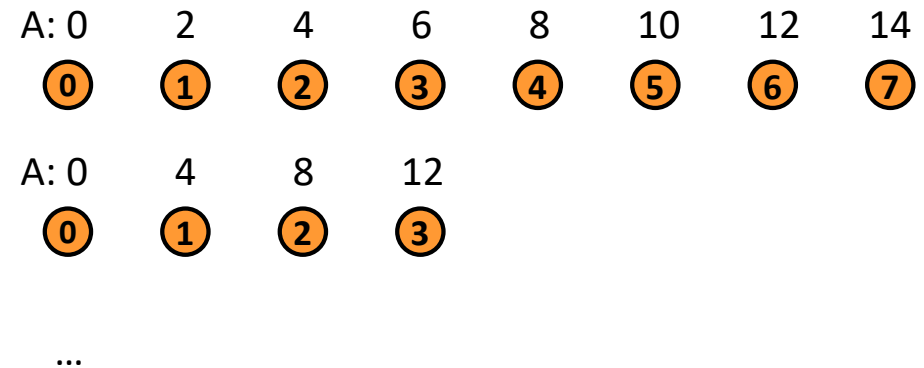
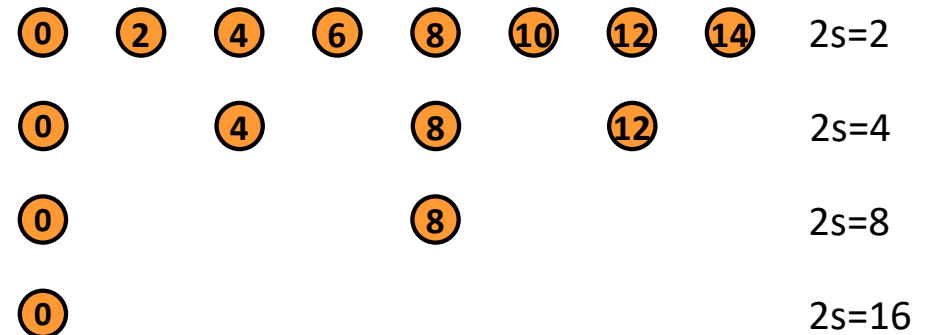
Reduction #2: Continuous Addressing → Less Thread Divergence

```
for (unsigned int s=1; s < blockDim.x; s *= 2){
    if (tid % (2*s) == 0) {
        sdata[tid] += sdata[tid + s];
    }
    __syncthreads();
}
```

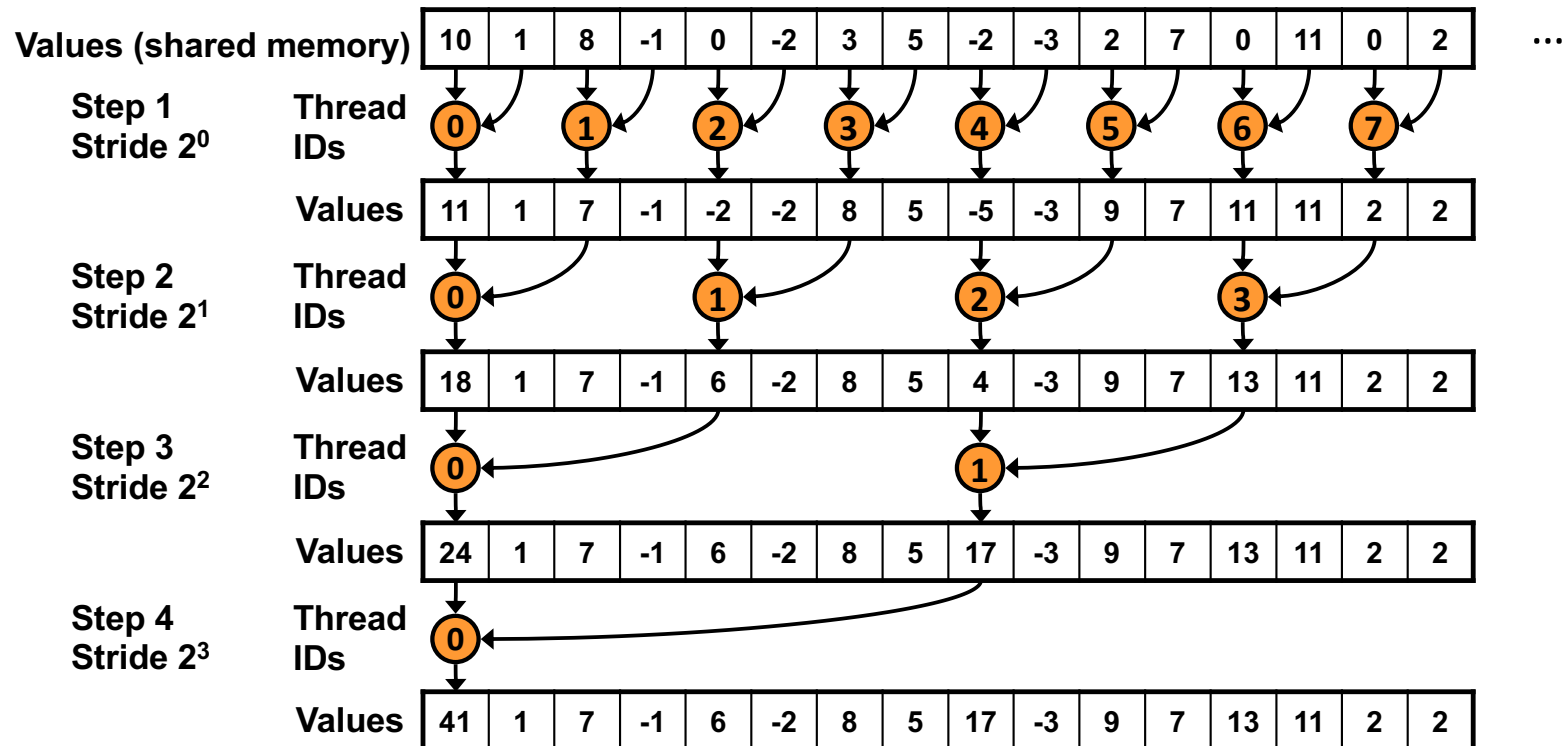
Replace divergent branch in inner loop with non-divergent branch from adjacent indices:

```
for (unsigned int s=1; s < blockDim.x; s *= 2){
    int index = 2 * s * tid; // A[index]

    if (index + s < blockDim.x) {
        sdata[index] += sdata[index + s];
    }
    __syncthreads();
}
```



Reduction #2: Interleaved Addressing → Less Thread Divergence

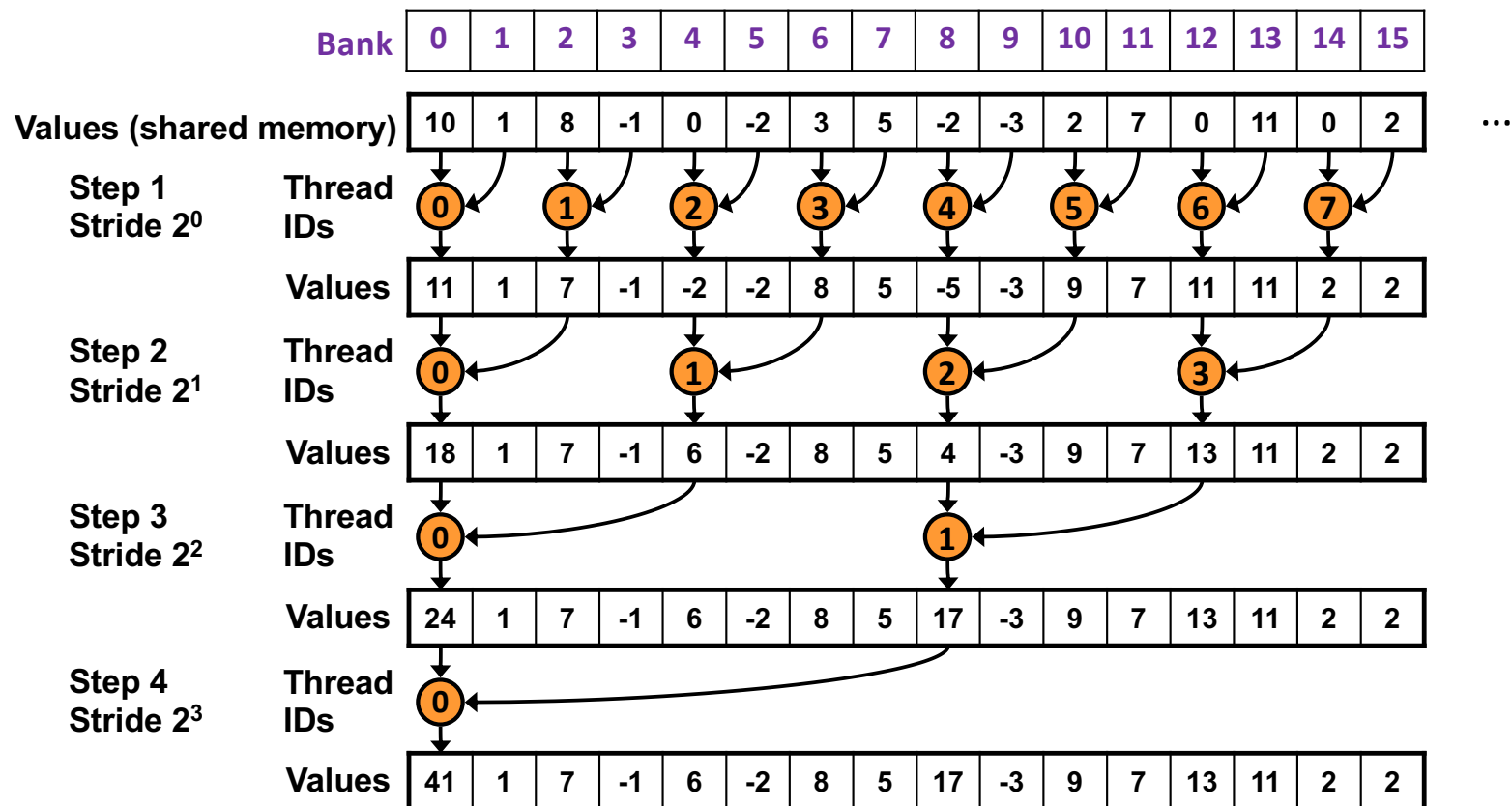


Successfully reduced thread divergence within a warp!

Performance for 4M element reduction

	Time (2^{22} ints)	Bandwidth	Step Speedup	Cumulative Speedup
Kernel 1: interleaved addressing with divergent branching	8.054 ms	2.083 GB/s		
Kernel 2: interleaved addressing with bank conflicts	3.456 ms	4.854 GB/s	2.33x	2.33x

Reduction #2: Less Thread Divergence. but Each Thread Takes Two Banks



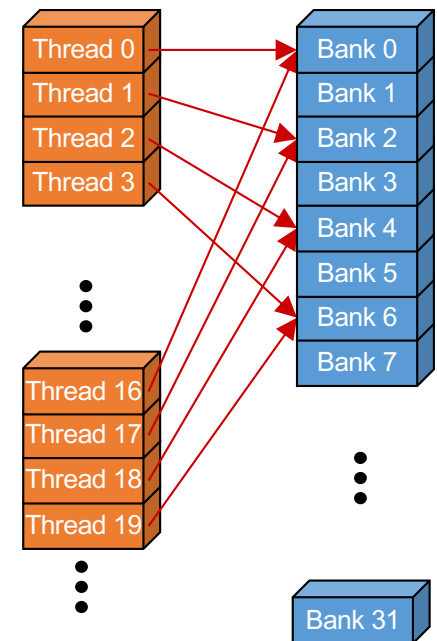
New Problem: Shared Memory Bank Conflicts: 2-way conflict

Bank Conflict Occurs When Stride Size Shares a Common Factor with 32

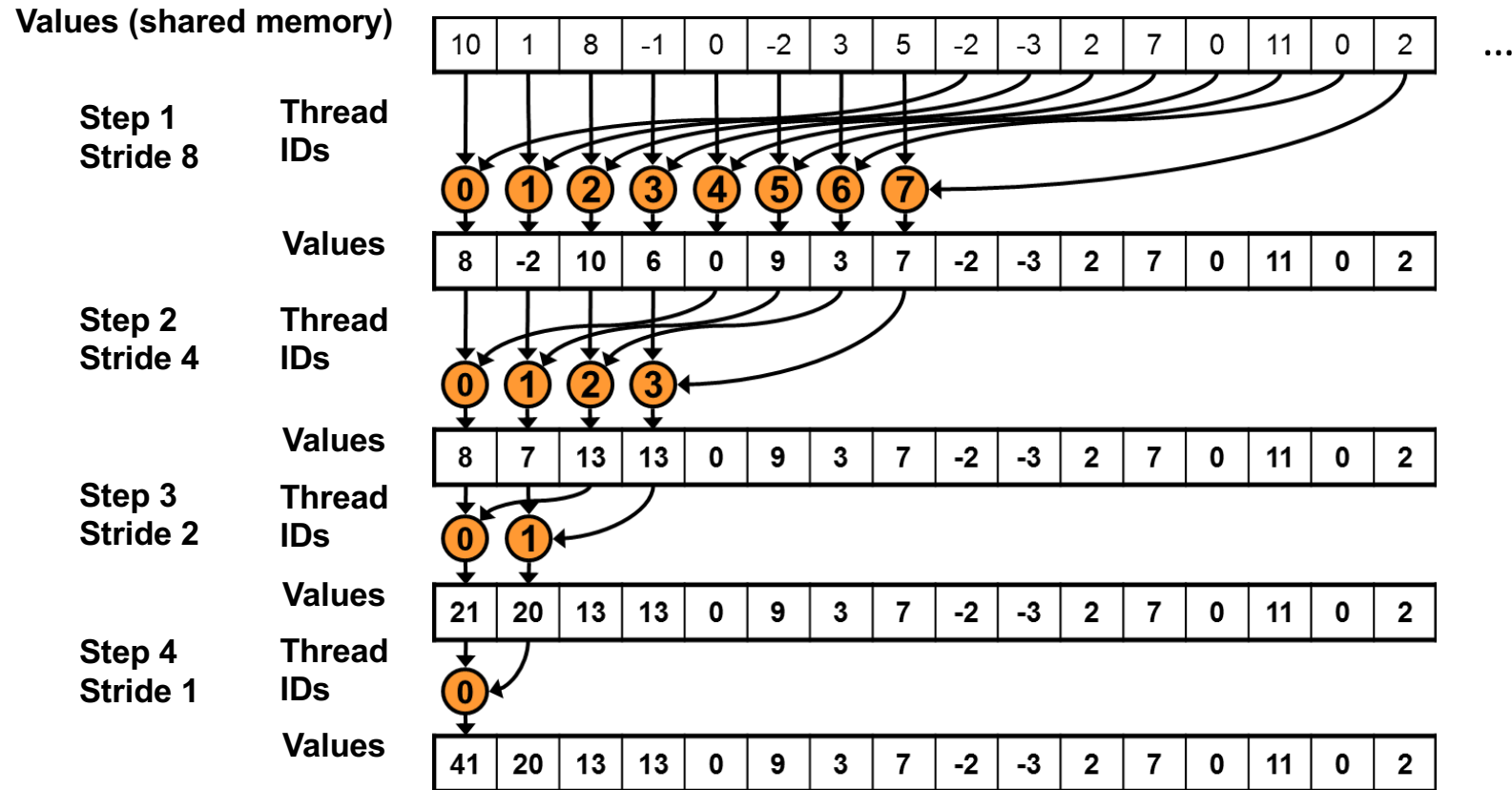
- Each thread loads two floats into ShMem:

```
int tid = threadIdx.x;  
int s = 2;  
shared[s*tid] = global[offset+s*tid];
```

- In our previous example:
 - Level 1: $s=2$ (2-way bank conflict)
 - Level 2: $s=4$ (4-way bank conflict)
 - Level 3: $s=8$ (8-way bank conflict)
 - ...
- To avoid bank conflict, stride size should be an odd number
 - Why? It has no common factor with 32 (# of banks)



Reduction #3: Sequential Addressing: One Bank per Thread



Sequential addressing is Shared Mem. **Bank-conflict free**

Reduction #3: Sequential Addressing

Just replace strided indexing in inner loop...

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {  
    int index = 2 * s * tid;  
  
    if (index < blockDim.x) {  
        sdata[index] += sdata[index + s];  
    }  
    __syncthreads();  
}
```

...with **reversed** loop and threadID-based indexing:

```
for (unsigned int s=blockDim.x/2; s>0; s>>=1) {  
    if (tid < s) {  
        sdata[tid] += sdata[tid + s];  
    }  
    __syncthreads();  
}
```



- First you go out 1/2 the block size to match a thread with its second operand
- Then you go out 1/4 of the block size
- Then you go out 1/8 of the block size
- Etc.

Performance for 4M element reduction

	Time (2^{22} ints)	Bandwidth	Step Speedup	Cumulative Speedup
Kernel 1: interleaved addressing with divergent branching	8.054 ms	2.083 GB/s		
Kernel 2: interleaved addressing with bank conflicts	3.456 ms	4.854 GB/s	2.33x	2.33x
Kernel 3: sequential addressing	1.722 ms	9.741 GB/s	2.01x	4.68x

Half of the Threads are Idle at the First Loop Iteration

- If we launch the kernel with the block size of 256, `reduce3<<<(N+255)/256, 256>>>`
 - Only half of the threads are doing useful work, and we immediately waste 128 threads!

```
for (unsigned int s=blockDim.x/2; s>0; s>>=1) {  
    if (tid < s) {  
        sdata[tid] += sdata[tid + s];  
    }  
    __syncthreads();  
}
```

- Solution: Assign more work to each thread!

Reduction #4: First Add on Load → Initial Per-thread Reduction

Replace single load:

```
// each thread loads one element from global to shared mem
unsigned int tid = threadIdx.x;
unsigned int i    = blockIdx.x*blockDim.x + threadIdx.x;
sdata[tid] = g_idata[i];
__syncthreads();
```

...With two loads and first add of the reduction:

```
// perform first level of reduction upon reading from
// global memory and writing to shared memory
unsigned int tid = threadIdx.x;
unsigned int i    = blockIdx.x*(blockDim.x*2) + threadIdx.x;
sdata[tid] = g_idata[i] + g_idata[i+blockDim.x];
__syncthreads();
```

Side-effect: number of blocks needed is **half of what it used to be**

Performance for 4M element reduction

	Time (2^{22} ints)	Bandwidth	Step Speedup	Cumulative Speedup
Kernel 1: interleaved addressing with divergent branching	8.054 ms	2.083 GB/s		
Kernel 2: interleaved addressing with bank conflicts	3.456 ms	4.854 GB/s	2.33x	2.33x
Kernel 3: sequential addressing	1.722 ms	9.741 GB/s	2.01x	4.68x
Kernel 4: first add during global load	0.965 ms	17.377 GB/s	1.78x	8.34x

What Else Can We Optimize?

- At 17 GB/s, we're far from bandwidth bound (>50 GB/s)
- Currently, we are neither compute-bound nor memory-bound → a likely bottleneck is instruction overhead
 - Ancillary instructions that are not loads, stores, or core arithmetic
 - In other words: **loop overhead** and **address arithmetic**
- Strategy: unroll loops

Unrolling the Last Warp

- As reduction proceeds, the number of “active” threads decreases
 - When $s \leq 32$, we have only one warp left
- Instructions executed in lockstep fashion within a warp – meaning that when $s \leq 32$:
 - We don’t need to `__syncthreads()`
 - We don’t need “`if (tid < s)`” because that’s a given

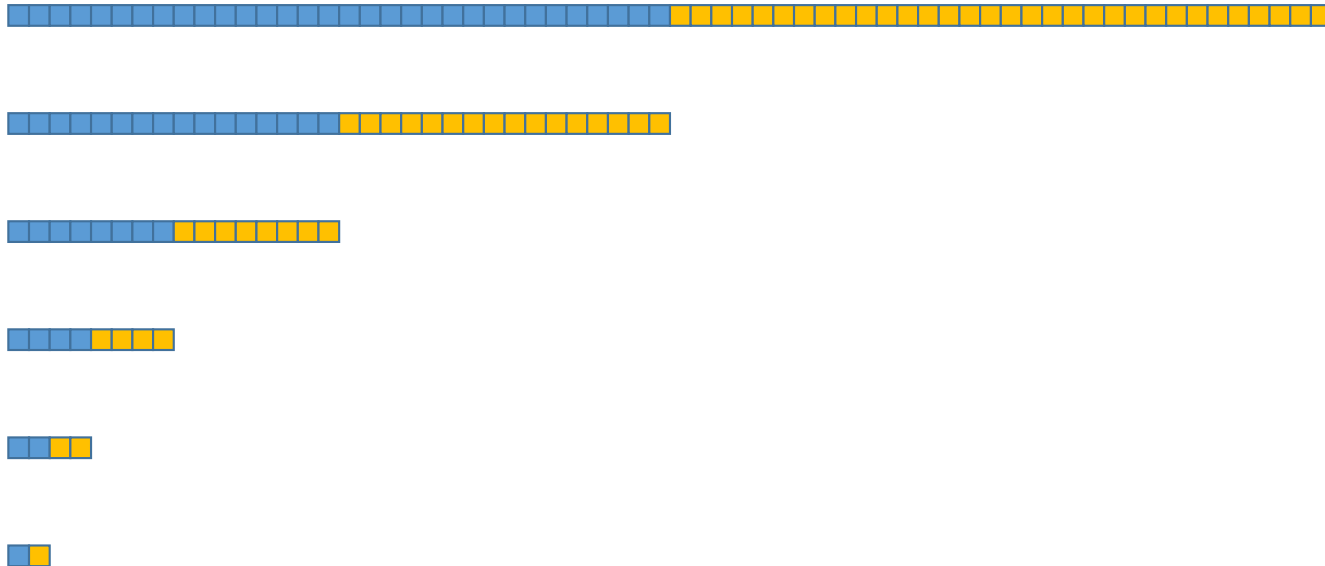
Key idea: unroll the last 6 iterations of the inner loop, which involve 32 or less threads

```
for (unsigned int s=blockDim.x/2; s>0; s>>=1) {  
    if (tid < s) {  
        sdata[tid] += sdata[tid + s];  
    }  
    __syncthreads();  
}
```

“unroll the last 6 iterations of the inner loop, which involve 32 or less threads”

Legend:

- - Array entry that gets updated, and accumulates result
- - Array entry that is read in order to update a “blue” entry



Reduction #5: Unroll the Last Warp

```
// and use later like this...
for (unsigned int s=blockDim.x/2; s>32; s>>=1) {
    if (tid < s)
        sdata[tid] += sdata[tid + s];
    __syncthreads();
}

if (tid < 32) warpReduce(sdata, tid);
```

This used to be:
`s>0`

```
__device__ void warpReduce(int* sdata, int tid) {
    sdata[tid] += sdata[tid + 32];
    if (tid < 16) sdata[tid] += sdata[tid + 16];
    if (tid < 8) sdata[tid] += sdata[tid + 8];
    if (tid < 4) sdata[tid] += sdata[tid + 4];
    if (tid < 2) sdata[tid] += sdata[tid + 2];
    if (tid < 1) sdata[tid] += sdata[tid + 1];
}
```

Note: This saves useless work in *all* warps, not just the last one!

Without unrolling, all warps execute every iteration of the for loop and if statement

Performance for 4M element reduction

	Time (2^{22} ints)	Bandwidth	Step Speedup	Cumulative Speedup
Kernel 1: interleaved addressing with divergent branching	8.054 ms	2.083 GB/s		
Kernel 2: interleaved addressing with bank conflicts	3.456 ms	4.854 GB/s	2.33x	2.33x
Kernel 3: sequential addressing	1.722 ms	9.741 GB/s	2.01x	4.68x
Kernel 4: first add during global load	0.965 ms	17.377 GB/s	1.78x	8.34x
Kernel 5: unroll last warp	0.536 ms	31.289 GB/s	1.8x	15.01x

Complete Unrolling at the “Block” Level

- If we knew the number of reduction consecutive steps (or equivalently, of threads in a block) at compile time, we could completely unroll the reduction
 - Luckily, the block size on G80 is limited by the GPU to 512 threads
 - 1024 on Fermi GPUs and newer
 - Also, we are sticking to power-of-2 block sizes
- Basic idea: unroll everything for a fixed block size
 - But we need to be generic – how can we unroll for block sizes that we don’t know at compile time?
- Use of templates can solve this issue...
 - CUDA supports C++ template parameters on device and host functions

Unrolling at Compile Time with C++ Templates

- Specify block size as a function template parameter
- The kernel is templated at the block size:

```
template <unsigned int blockSize>  
__global__ void reduce6(int *g_idata, int *g_odata)
```

Reduction #6: Completely Unrolled

This is the key part of the kernel

```
If constexpr(blockSize >= 512) {  
    if(tid < 256){ sdata[tid] += sdata[tid + 256]; } __syncthreads();}  
If constexpr(blockSize >= 256) {  
    if(tid < 128){ sdata[tid] += sdata[tid + 128]; } __syncthreads();}  
If constexpr(blockSize >= 128) {  
    if(tid < 64){ sdata[tid] += sdata[tid + 64]; } __syncthreads();}  
  
if(tid < 32) warpReduce<blockSize>(sdata, tid); // last warp only
```

This is a helper function (device only)

```
template <unsigned int blockSize>  
__device__ void warpReduce(int* sdata, int tid) {  
    if constexpr(blockSize >= 64) sdata[tid] += sdata[tid + 32];  
    if constexpr(blockSize >= 32) sdata[tid] += sdata[tid + 16];  
    if constexpr(blockSize >= 16) sdata[tid] += sdata[tid + 8];  
    if constexpr(blockSize >= 8) sdata[tid] += sdata[tid + 4];  
    if constexpr(blockSize >= 4) sdata[tid] += sdata[tid + 2];  
    if constexpr(blockSize >= 2) sdata[tid] += sdata[tid + 1];  
}
```

- All code in **RED** will be evaluated at compile time. Results in a very efficient inner loop.
- For Fermi and newer CC, you'd have one more `if constexpr` statement that covers the case when `blockSize >= 1024`
- You can call the **warpReduce** function only when you got to one warp. Reason: you don't have to synchronize at that point.

Invoking Template Kernels

- Don't we still need block size at compile time?
 - There is a way out, use a switch statement for 10 possible block sizes:

```
switch (threads) {  
case 512:  
    reduce6<512><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;  
case 256:  
    reduce6<256><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;  
case 128:  
    reduce6<128><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;  
case 64:  
    reduce6< 64><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;  
case 32:  
    reduce6< 32><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;  
case 16:  
    reduce6< 16><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;  
case 8:  
    reduce6<  8><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;  
case 4:  
    reduce6<  4><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;  
case 2:  
    reduce6<  2><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;  
case 1:  
    reduce6<  1><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;  
}
```

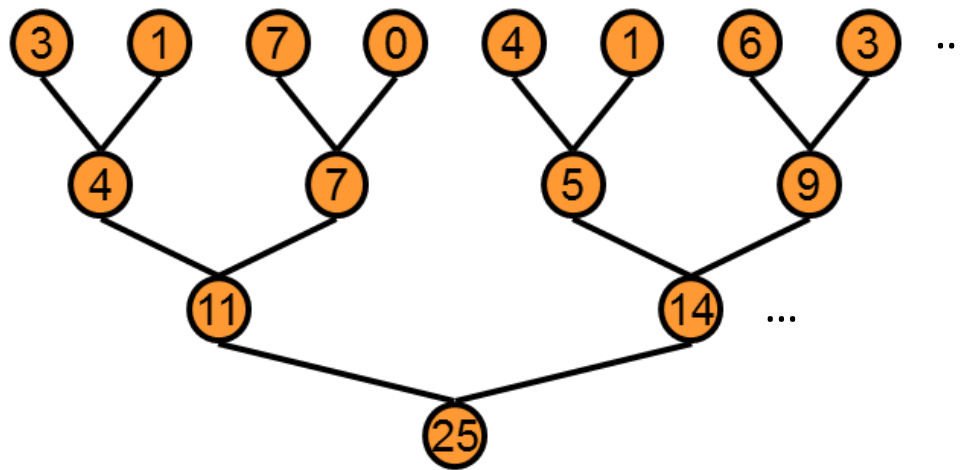
This is code on the host, calling the appropriate kernel

Performance for 4 Million element reduction

	Time (2^{22} ints)	Bandwidth	Step Speedup	Cumulative Speedup
Kernel 1: interleaved addressing with divergent branching	8.054 ms	2.083 GB/s		
Kernel 2: interleaved addressing with bank conflicts	3.456 ms	4.854 GB/s	2.33x	2.33x
Kernel 3: sequential addressing	1.722 ms	9.741 GB/s	2.01x	4.68x
Kernel 4: first add during global load	0.965 ms	17.377 GB/s	1.78x	8.34x
Kernel 5: unroll last warp	0.536 ms	31.289 GB/s	1.8x	15.01x
Kernel 6: completely unrolled	0.381 ms	43.996 GB/s	1.41x	21.16x

Time Complexity and Work Efficiency of Parallel Reduction

- Assume that the number of elements in array is N



N works



$N/2$ works



$N/4$ works



...

Time complexity = $\text{Log}(N)$ stages

- Total work = $N + N/2 + N/4 + \dots + 1 = O(N/(1-1/2)) = O(N)$
- The algorithm is *work-efficient* as it does not go beyond the sequential implementation

$$\sum_{i=1}^n a_1 r^{i-1} = S_n = \frac{a_1(1 - r^n)}{1 - r}, \quad r \neq 1$$

What About Cost?

- Cost of a parallel algorithm is *threads* $\times$ *time* complexity
 - Allocates $O(N)$ threads
 - Time complexity is $O(\log N)$, so *cost* is $O(N \log N)$: **not cost efficient!**
- Solution: why don't we just allocate $O(N/\log N)$ threads? – Brent's theorem
 - Each thread does $O(\log N)$ *sequential work* (local initial reduction) before entering parallel reduction
 - Then all $O(N/\log N)$ threads cooperate for $O(\log N)$ stages
 - Cost = $O((N/\log N) * \log N) = O(N) \rightarrow$ cost efficient
- Sometimes called *algorithm cascading*
 - Basically, each thread runs a local reduction on several elements first
 - In practice, this can give significant speedup due to better utilization of per-thread registers

Reduction #7: Multiple Adds / Thread

Replace load and add of two elements

```
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*(blockDim.x*2) + threadIdx.x;
sdata[tid] = g_idata[i] + g_idata[i+blockDim.x];
__syncthreads();
```

with a while loop to add as many as necessary:

```
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*(blockSize*2) + threadIdx.x;
unsigned int gridSize = blockSize*2*gridDim.x;
sdata[tid] = 0;

while (i < n) {
    sdata[tid] += g_idata[i] + g_idata[i+blockSize];
    i += gridSize;
}
__syncthreads();
```

Note: gridSize loop stride to maintain coalescing!

Performance for 4 Million element reduction

	Time (2 ²² ints)	Bandwidth	Step Speedup	Cumulative Speedup
Kernel 1: interleaved addressing with divergent branching	8.054 ms	2.083 GB/s		
Kernel 2: interleaved addressing with bank conflicts	3.456 ms	4.854 GB/s	2.33x	2.33x
Kernel 3: sequential addressing	1.722 ms	9.741 GB/s	2.01x	4.68x
Kernel 4: first add during global load	0.965 ms	17.377 GB/s	1.78x	8.34x
Kernel 5: unroll last warp	0.536 ms	31.289 GB/s	1.8x	15.01x
Kernel 6: completely unrolled	0.381 ms	43.996 GB/s	1.41x	21.16x
Kernel 7: multiple elements per thread	0.268 ms	62.671 GB/s	1.42x	30.04x

Kernel 7 on 32M elements: 73 GB/s!

Final Kernel Implementation

```
template <unsigned int blockSize>
__device__ void warpReduce(int *sdata, unsigned int tid) {
    if (blockSize >= 64) sdata[tid] += sdata[tid + 32];
    if (blockSize >= 32) sdata[tid] += sdata[tid + 16];
    if (blockSize >= 16) sdata[tid] += sdata[tid + 8];
    if (blockSize >= 8) sdata[tid] += sdata[tid + 4];
    if (blockSize >= 4) sdata[tid] += sdata[tid + 2];
    if (blockSize >= 2) sdata[tid] += sdata[tid + 1];
}

template <unsigned int blockSize>
__global__ void reduce7(int *g_idata, int *g_odata, unsigned int n) {
    extern __shared__ int sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x*(blockSize*2) + tid;
    unsigned int gridSize = blockSize*2*gridDim.x;
    sdata[tid] = 0;

    while (i < n) { sdata[tid] += g_idata[i] + g_idata[i+blockSize]; i += gridSize; }
    __syncthreads();

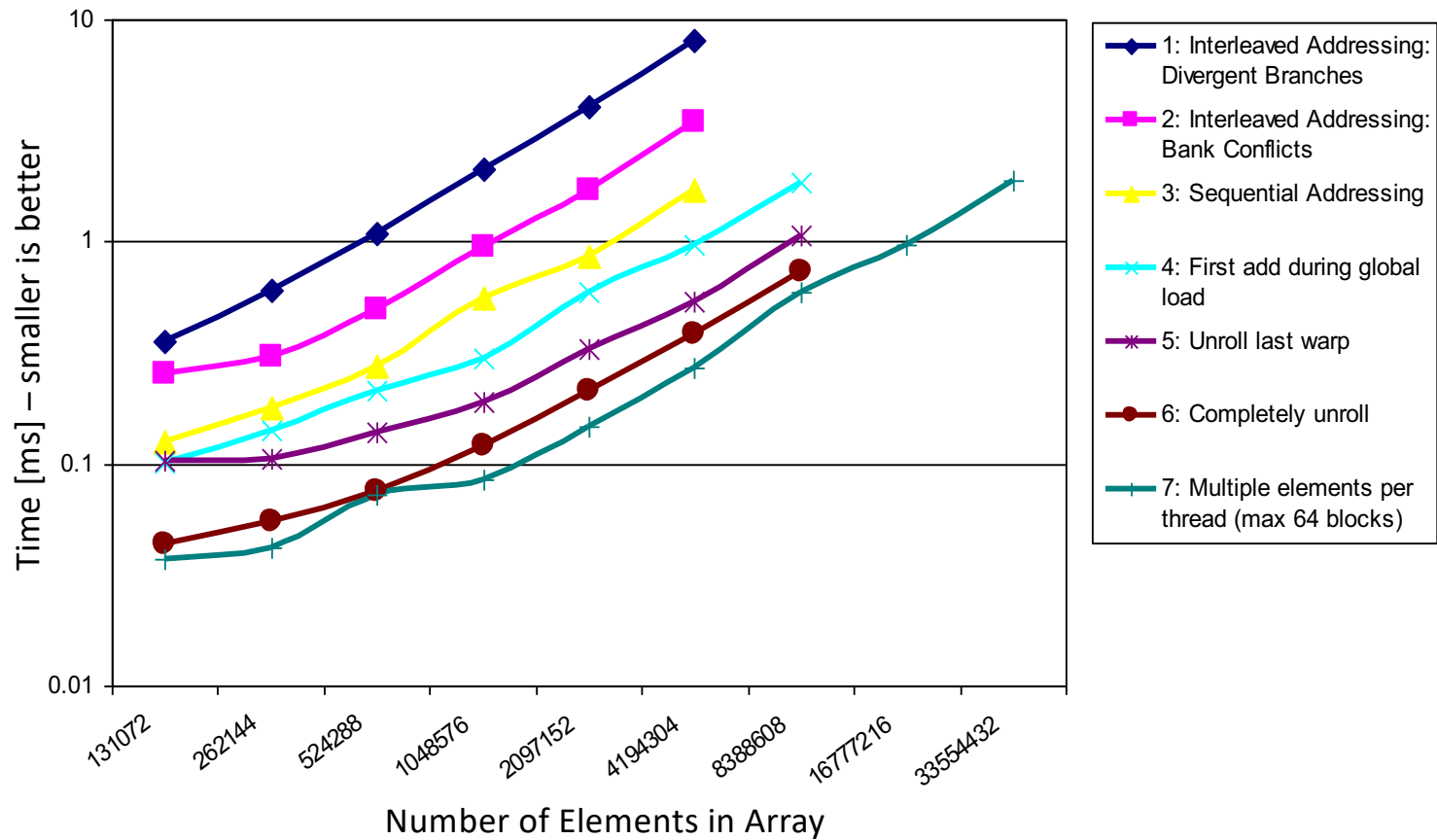
    if (blockSize >= 512) { if (tid < 256) { sdata[tid] += sdata[tid + 256]; } __syncthreads(); }
    if (blockSize >= 256) { if (tid < 128) { sdata[tid] += sdata[tid + 128]; } __syncthreads(); }
    if (blockSize >= 128) { if (tid < 64) { sdata[tid] += sdata[tid + 64]; } __syncthreads(); }

    if (tid < 32) warpReduce(sdata, tid);
    if (tid == 0) g_odata[blockIdx.x] = sdata[0];
}
```

Final Kernel Implementation – Invoke

```
switch (threads) {
case 512:
    reduce6<512><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
case 256:
    reduce6<256><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
case 128:
    reduce6<128><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
case 64:
    reduce6< 64><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
case 32:
    reduce6< 32><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
case 16:
    reduce6< 16><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
case 8:
    reduce6<  8><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
case 4:
    reduce6<  4><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
case 2:
    reduce6<  2><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
case 1:
    reduce6<  1><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
}
```

Performance Comparison



Sources of Efficiency Improvement

- Algorithmic optimizations
 - Changes to addressing, algorithm cascading
 - 11.84x speedup, combined
- Code optimizations
 - Loop unrolling
 - 2.54x speedup, combined

Lessons Learned from Array Reduction

- Start with a simple, naïve implementation and then iterate the following four steps
 - Parallelize
 - Optimize
 - Deploy
 - Assess
- Understand CUDA performance characteristics
 - Thread divergence
 - Bank conflicts
 - Latency hiding
- Almost all CUDA kernel programming follows the same paradigm

